

Nota: Este material complementar, disponível em <https://prettore.github.io/lectures.html> representa uma cópia resumida de conteúdos bibliográficos disponíveis gratuitamente na Internet.

Variáveis, Constantes e Atribuições

Introdução.....	1
Declaração de Variáveis e Tipos Primitivos.....	1
Regras de Nomenclatura e Escopo.....	2
Inicialização de Variáveis.....	3
Constantes: Definição e Uso.....	3
Operador de Atribuição Simples e Composto.....	4
Conclusão.....	5
Referências.....	5

Introdução

As variáveis, constantes e atribuições são conceitos fundamentais na programação de computadores, formando a base para o armazenamento e manipulação de dados em qualquer linguagem de programação. Esses elementos permitem que os programadores criem soluções dinâmicas e flexíveis para diversos problemas computacionais. Compreender esses conceitos é essencial para qualquer estudante de Ciência da Computação, pois eles são utilizados em praticamente todos os programas, desde os mais simples até os mais complexos. Este material explora os principais aspectos relacionados a variáveis, constantes e atribuições, abordando sua definição, tipos, regras de nomenclatura, escopo e aplicações práticas.

Declaração de Variáveis e Tipos Primitivos

Uma variável é um espaço reservado na memória do computador para armazenar dados que podem ser modificados durante a execução de um programa. Quando declaramos uma variável, estamos essencialmente solicitando ao sistema operacional que reserve um espaço na memória RAM para armazenar um determinado tipo de dado.

A declaração de variáveis segue uma sintaxe específica que varia de acordo com a linguagem de programação utilizada. Em geral, a declaração inclui o tipo de dado que a variável irá armazenar, seguido pelo nome (identificador) da variável. Por exemplo, em C:

```
int idade;
float altura;
char inicial;
```

Os tipos primitivos (ou básicos) são os blocos fundamentais de construção para armazenamento de dados em programação. Eles variam ligeiramente entre as linguagens, mas geralmente incluem:

- Tipos Numéricos:
 - a. Inteiros (int): Armazenam números inteiros, positivos ou negativos, sem parte decimal.
 - b. Reais (float, double): Armazenam números com parte decimal, também conhecidos como números de ponto flutuante.
- Tipo Caractere (char): Armazena um único caractere, como uma letra, número ou símbolo.
- Tipo Booleano (bool): Armazena valores lógicos, verdadeiro (true) ou falso (false).
- Tipo String: Embora não seja considerado primitivo em todas as linguagens, representa uma sequência de caracteres.

Cada tipo primitivo ocupa um espaço específico na memória e tem limites para os valores que pode armazenar. Por exemplo, em muitas implementações, um `int` ocupa 4 bytes (32 bits) e pode armazenar valores entre -2.147.483.648 e 2.147.483.647.

Regras de Nomenclatura e Escopo

A escolha de nomes para variáveis segue regras específicas que devem ser respeitadas para que o programa funcione corretamente. Essas regras variam ligeiramente entre as linguagens de programação, mas alguns princípios gerais incluem:

- Os nomes de variáveis devem começar com uma letra ou um underscore (`_`), nunca com um número ou caractere especial.
- Após o primeiro caractere, podem ser usados letras, números e underscores.
- Não são permitidos espaços ou caracteres especiais como `@`, `#`, `$`, `%`, etc.
- Não é possível usar palavras reservadas da linguagem (como `if`, `while`, `for`, etc.) como nomes de variáveis.
- A maioria das linguagens diferencia maiúsculas de minúsculas (case-sensitive), portanto `"idade"` e `"Idade"` seriam variáveis diferentes.

Além das regras sintáticas, existem convenções de nomenclatura que ajudam a tornar o código mais legível e manutenível:

- `camelCase`: Primeira palavra em minúscula, demais palavras iniciando com maiúscula (ex: `idadeAluno`).
- `snake_case`: Palavras separadas por underscore, todas em minúsculas (ex: `idade_aluno`).
- `PascalCase`: Todas as palavras iniciando com maiúscula (ex: `IdadeAluno`).

O escopo de uma variável define onde ela pode ser acessada dentro do programa. Os principais tipos de escopo são:

- **Escopo Global**: A variável é declarada fora de qualquer função ou bloco e pode ser acessada em qualquer parte do programa.
- **Escopo Local**: A variável é declarada dentro de uma função ou bloco e só pode ser acessada dentro desse contexto.
- **Escopo de Bloco**: Em linguagens como C++ e Java, variáveis declaradas dentro de blocos (como loops ou condicionais) só existem dentro desses blocos.

O escopo adequado é fundamental para evitar conflitos de nomes e garantir que as variáveis sejam utilizadas apenas onde necessário, contribuindo para a segurança e eficiência do código.

Inicialização de Variáveis

A inicialização de variáveis é o processo de atribuir um valor inicial a uma variável no momento de sua declaração. Embora nem todas as linguagens exijam inicialização imediata, é considerada uma boa prática de programação, pois evita o uso de valores indeterminados ou "lixo" que podem estar presentes na memória.

Existem diferentes formas de inicializar variáveis:

- **Inicialização direta**: O valor é atribuído no momento da declaração.

```
int idade = 25;  
float altura = 1.75;
```

- Inicialização por expressão: O valor inicial é calculado a partir de uma expressão.

```
int area = largura * altura;
float media = (nota1 + nota2) / 2;
```
- Inicialização por entrada do usuário: O valor é obtido através de uma função de entrada.

```
int idade = scanf("%d", &idade); // Em C
String nome = input.nextLine(); // Em Java
```

Em algumas linguagens, variáveis não inicializadas recebem valores padrão (como 0 para números, false para booleanos, etc.), enquanto em outras, o valor inicial é indeterminado. Usar variáveis não inicializadas pode levar a comportamentos imprevisíveis e bugs difíceis de detectar, por isso a inicialização adequada é essencial para um código robusto.

Constantes: Definição e Uso

Uma constante é um tipo especial de variável cujo valor, uma vez atribuído, não pode ser alterado durante a execução do programa. Constantes são utilizadas para representar valores fixos que não devem mudar, como o valor de π (pi), a aceleração da gravidade, ou configurações do sistema.

A declaração de constantes varia entre as linguagens de programação:

- Em C/C++: `#define PI 3.14159` ou `const double PI = 3.14159;`
- Em Java: `final double PI = 3.14159;`
- Em Python: `PI = 3.14159` (por convenção, constantes são escritas em maiúsculas)

As constantes oferecem várias vantagens:

- Segurança: Evitam modificações acidentais de valores que deveriam permanecer fixos.
- Legibilidade: Tornam o código mais compreensível ao substituir "números mágicos" por nomes significativos.
- Manutenção: Facilitam alterações futuras, pois o valor precisa ser modificado em apenas um lugar.
- Otimização: Permitem que o compilador realize otimizações específicas.

É recomendado usar constantes sempre que um valor não deve mudar durante a execução do programa e aparece múltiplas vezes no código. Por convenção, os nomes de constantes são geralmente escritos em letras maiúsculas com palavras separadas por underscores (SNAKE_CASE_MAIÚSCULO), como `PI`, `MAX_ALUNOS`, ou `TAXA_JUROS`.

Operador de Atribuição Simples e Composto

O operador de atribuição é utilizado para atribuir valores a variáveis. O operador de atribuição simples é representado pelo símbolo de igual (=) na maioria das linguagens de programação. Ele copia o valor do lado direito para a variável do lado esquerdo.

- `idade = 25;` // Atribui o valor 25 à variável idade
- `nome = "João";` // Atribui a string "João" à variável nome

É importante não confundir o operador de atribuição (=) com o operador de igualdade (==), que é usado para comparações.

Os operadores de atribuição compostos combinam uma operação aritmética com a atribuição, tornando o código mais conciso. Os principais operadores compostos são:

- `+=` : Adiciona e atribui (`x += 5` equivale a `x = x + 5`)
- `-=` : Subtrai e atribui (`x -= 3` equivale a `x = x - 3`)
- `*=` : Multiplica e atribui (`x *= 2` equivale a `x = x * 2`)
- `/=` : Divide e atribui (`x /= 4` equivale a `x = x / 4`)
- `%=` : Calcula o resto da divisão e atribui (`x %= 3` equivale a `x = x % 3`)

Esses operadores não apenas tornam o código mais compacto, mas também podem ser mais eficientes em termos de execução, pois a variável é acessada apenas uma vez.

Em linguagens que suportam programação orientada a objetos, a atribuição de objetos merece atenção especial, pois pode envolver cópia de referência (shallow copy) ou cópia de valores (deep copy), dependendo da implementação da linguagem e do tipo de objeto.

Conclusão

Variáveis, constantes e atribuições são conceitos fundamentais que formam a base da programação de computadores. Eles permitem o armazenamento, manipulação e controle de dados, possibilitando a criação de programas dinâmicos e interativos.

A compreensão adequada desses conceitos é essencial para o desenvolvimento de habilidades de programação sólidas. Saber como declarar variáveis corretamente, escolher os tipos de dados apropriados, seguir as regras de nomenclatura, entender o escopo, utilizar constantes quando necessário e aplicar operadores de atribuição de forma eficiente são competências que todo programador deve dominar.

À medida que avançamos para tópicos mais complexos como estruturas de controle, funções e estruturas de dados, esses conceitos básicos continuarão sendo aplicados e expandidos. Portanto, é fundamental construir uma base sólida nessas áreas para facilitar o aprendizado futuro e o desenvolvimento de soluções computacionais eficientes e robustas.

Referências

-  Livros e Apostilas
 - CORMEN, T. H. Introduction to Algorithms. MIT Press.
 - GOODRICH, M. Data Structures and Algorithms in Python.
 - Tenenbaum, A. M. Estruturas de Dados e Algoritmos em C
 - P. Feofiloff. Algoritmos em Linguagem C. Campus-Elsevier, 1a. edição, 2009
 - H. M. Deitel, P. J. Deitel. C - Como Programar, 6a. edição, Pearson Education, 2011.
 - B. W. Kernighan, D. M. Ritchie. The C Programming Language, 2a. edição, Prentice-Hall, 1988 [Tradução: C - A Linguagem de Programação. Editora Campus, 1989].
 - J. L. Szwarcfiter, L. Markenzon. Estruturas de Dados e seus Algoritmos, 3a. edição, Editora LTC, 2010.
 - W. Celes, R. Cerqueira, J.L. Rangel. Introdução a Estruturas de Dados, 1a. edição, Editora Campus, 2004.
 - N. Ziviani. Projeto de Algoritmos com Implementações em Pascal e C, 3a. edição, Editora Cengage Learning, 2011.
 - T. Cormen, C. Leiserson, R. Rivest, C. Stein. Algoritmos - Teoria e Prática, 3a. edição, Editora Campus, 2012.
 - R. Sedgewick, K. Wayne. Algorithms, 4a. edição, Addison -Wesley, 2011.
 - A. Kelley, I. Pohl. A Book on C, 4a. edição, Addison Wesley, 1998.
-  Recursos Online
 - Asimov Academy - O que são variáveis e constantes na programação? - <https://hub.asimov.academy/blog/variaveis-constantes-programacao/>
 - TreinaWeb - Variáveis e constantes na programação - <https://www.treinaweb.com.br/blog/variaveis-e-constantes-na-programacao>

- IBM - Constantes e Variáveis - <https://www.ibm.com/docs/pt-br/cloud-pak-system-w3500/2.3.3?topic=language-constants-variables>
- DIO - O que são Variáveis e Constantes em programação? - <https://www.dio.me/articles/o-que-sao-variaveis-e-constantes-em-programacao>
- Embarcados - Artigo: Variáveis e Constantes em Algoritmos - <https://embarcados.com.br/variaveis-e-constantes/>
- DevsChannel - Variáveis e constantes - <https://devschannel.com/logica-de-programacao/variaveis-e-constantes>
- GeeksforGeeks – Variables and Keywords in Programming
- 📺 Vídeos e Cursos
 - Curso em Vídeo – Variáveis e Tipos Primitivos no YouTube
 - MIT OpenCourseWare – Introduction to Computer Science and Programming

Isenção de Responsabilidade:

Os autores deste documento não reivindicam a autoria do conteúdo original compilado das fontes mencionadas. Este documento foi elaborado para fins educativos e de referência, e todos os créditos foram devidamente atribuídos aos respectivos autores e fontes originais.

Qualquer utilização comercial ou distribuição do conteúdo aqui compilado deve ser feita com a devida autorização dos detentores dos direitos autorais originais. Os compiladores deste documento não assumem qualquer responsabilidade por eventuais violações de direitos autorais ou por quaisquer danos decorrentes do uso indevido das informações contidas neste documento.

Ao utilizar este documento, o usuário concorda em respeitar os direitos autorais dos autores originais e isenta os compiladores de qualquer responsabilidade relacionada ao conteúdo aqui apresentado.